



How Does Killing Surviving Mutants Help Detect Real Bugs with Assertion Generation? A Controlled Experiment

HANG DU, University of California at Irvine, USA

VIJAY KRISHNA PALEPU, Microsoft, USA

JAMES A. JONES, University of California at Irvine, USA

Killing surviving mutants is a central activity of mutation testing. This activity is motivated by the coupling-effect hypothesis: tests that expose simple artificial faults can also detect more complex, previously unseen real bugs. Despite these claimed benefits, automated studies have not directly measured the causal impact of mutant killing on real-bug detection. This limitation stems from open-ended mutant-killing strategies and a fundamental evaluation asymmetry that obscures causal attribution. In this work, we present the first large-scale controlled experiment that directly measures whether killing surviving mutants, without knowledge of future real bugs, would have enabled their detection. We model mutant killing as a selective, incremental process under realistic budget constraints, and we restrict test improvements to assertion augmentation, thereby targeting the subset of surviving mutants whose survival stems from revealability gaps in existing tests. This restriction enables precise attribution of each test augmentation to a specific mutant-killing action. To support the experiment, we design a fully automated, fault-based assertion-augmentation technique that operates uniformly on mutants and real bugs and integrate it into Defects4J. Our controlled experiment yields several key empirical insights: (1) Across 642 Defects4J bugs, we find that 104 bugs would become detectable by adding an additional assertion to an existing passing, non-triggering test. (2) When coupling exists, a real bug is, on average, coupled with 21 surviving mutants, through which mutant killing can produce triggering tests. This number is substantially higher than the average of two mutants reported in prior studies. In those studies, coupling is inferred solely from documented bug-fixing tests rather than from tests derived via mutant killing. (3) Among these bugs, 63 of the 104 are detectable through a principled mutant-killing (test augmentation) process. Notably, killing a randomly selected 30% of relevant surviving mutants, using only one assertion per mutant, suffices to detect 84.5% of these bugs. (4) By substituting mutants with real bugs and comparing their resulting assertion augmentation outputs, we find that real bugs induce broader behavioral effects than mutants, affecting more memory state locations, variables, and tests. (5) When mutation-derived assertions detect real bugs, they validate program outputs that overlap with, and are often strict subsets of, those affected by the real bugs. This offers a mechanistic explanation for why killing simple mutants can enable the detection of more complex real bugs.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**.

Additional Key Words and Phrases: mutation testing, coupling effect, bug detection

ACM Reference Format:

Hang Du, Vijay Krishna Palepu, and James A. Jones. 2026. How Does Killing Surviving Mutants Help Detect Real Bugs with Assertion Generation? A Controlled Experiment. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA141 (October 2026), 24 pages. <https://doi.org/10.1145/3832232>

Authors' Contact Information: [Hang Du](#), University of California at Irvine, Irvine, USA, hdu5@uci.edu; [Vijay Krishna Palepu](#), Microsoft, Mountain View, USA, vijay.palepu@microsoft.com; [James A. Jones](#), University of California at Irvine, Irvine, USA, jajones@uci.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA141

<https://doi.org/10.1145/3832232>

1 Introduction

Killing surviving mutants is a central activity in improving test suites in mutation testing [54, 60, 66]. In this process, developers strengthen test suites by writing new tests to detect artificial faults (mutations) that are not detected by existing tests. This practice is appealing because it is grounded in a long-standing intuition: tests designed to reveal simple artificial faults may also help detect or prevent more complex, previously unseen real bugs. This intuition is commonly motivated by the *coupling-effect hypothesis*, introduced by DeMillo *et al.* alongside mutation testing [15], which posits that tests capable of revealing simple faults also tend to reveal more complex ones.

Despite its central role in mutation testing, the benefits of mutant killing—specifically its claimed ability to help detect or prevent future real bugs—have not been directly measured. There are two fundamental reasons for this. **First, mutant killing is an open-ended, selective, and subjective process:** in practice, developers resolve only a subset of surviving mutants due to limited time, resources, and perceived actionability constraints [55]. And, for each selected mutant, developers may choose among many plausible test designs [17, 55], which are not uniquely determined by the mutant and can vary substantially across developers, projects, and contexts. **Second, there exists a fundamental evaluation asymmetry:** when tests are written to kill mutants, we typically do not know whether those tests would have detected future real bugs that never occurred. Conversely, when a real bug is discovered and fixed with a bug-triggering test, it remains unknown whether such a test could plausibly have been written earlier through mutant-killing activities. Consequently, existing studies do not directly observe or test what would happen if developers killed certain mutants; instead, they infer the outcome by assuming that mutants “coupled” to real bugs would induce bug-fixing tests that detect those bugs [35, 40, 54].

In this work, we directly study the benefits of mutant killing by asking a simple but previously unanswered question: *If developers kill surviving mutants without any knowledge of a real bug, would those actions have helped detect that real bug had it occurred later?*

To answer this question, we model mutant killing as a selective, targeted intervention: (1) developers resolve a subset of surviving mutants and apply a single test improvement per mutant; and (2) mutant killing is performed in a targeted manner—strengthening tests only where necessary to kill specific mutants. For this purpose, we deliberately constrain mutant killing to assertion augmentation (alternatively known as “amplification”) and develop a technique that kills mutants by adding assertions to existing tests, without modifying test prefixes. Although idealized relative to real developers’ mutant-killing behavior—which may involve broader and more diverse code changes—the generated assertions precisely capture the exact memory-state variations induced by the mutation or real bug, which allows a comparison of assertions guided by either type of fault.

To further investigate *why* mutant killing succeeds or fails in detecting real bugs, we extend our analysis by generating augmentations directly from real faults and comparing them with mutation-derived augmentations. This comparison reveals where mutation-induced impacts overlap with, or differ from, real-bug-revealing behaviors at the assertion level. Together, our results provide direct empirical evidence for the coupling effect at the level of individual mutant kills and offer a plausible explanation for how killing mutants translates into improved real-bug detection in practice.

The main contributions of this paper are:

- We design and implement a fully automated, fault-based assertion-augmentation technique that generates localized assertions for mutants and bugs, integrated into Defects4J [34].
- We propose a controlled experimental framework for evaluating the benefits of mutant killing by modeling selective, targeted killing of surviving mutants *without knowledge of real bugs*, and directly measuring whether such actions could have detected future real bugs.

- We provide the first direct and automated empirical evidence that killing surviving mutants—specifically, those killable through assertion augmentation—can improve real-bug detection, quantify this effect under varying mutant-killing intensities, and provide multifaceted intermediate insights into the mutant-killing process.
- We directly measure the effects of substituting mutants with real bugs by comparing fault-based augmentations derived from each, revealing how mutation-derived assertions detect real bugs and offering a plausible explanation of the coupling effect.

2 Background, Motivation and Challenges

Mutation analysis and testing are widely used techniques for evaluating and improving test suites [32, 52]. The approach injects artificial faults, *i.e.*, mutations, into a program and evaluates whether a test suite can detect the resulting faulty program, *i.e.*, mutant. By repeatedly injecting mutations and executing the test suite, mutation analysis exposes test weaknesses and highlights opportunities for improvement by identifying undetected, or surviving mutants. Since its inception, the *mutation score*, *i.e.*, the ratio of killed mutants to all non-equivalent mutants, has become the dominant proxy for assessing test-suite effectiveness in research [32, 49, 52, 71].

Despite the widespread use of mutation score as a test adequacy criterion, the problem of killing surviving mutants has received comparatively less research attention, even though it is viewed as a practical means to improve tests and prevent future real bugs [60]. Specifically, a recent survey of 104 mutation-testing users reports that most practitioners use mutation testing to improve or create tests that kill specific mutants (83.7% and 76.0%, respectively), while fewer use mutation score as a stopping criterion (48.1%) [60]. Another industry study shows that mutation testing’s value depends less on the score itself and more on how surviving mutants are surfaced and translated into actionable test improvements [66]. This view is echoed by studies at Google [55] and Meta [29].

Mutant Killing Scenarios. Mutant killing is a unique form of test improvement that follows a principled workflow with several defining characteristics. This workflow has three defining characteristics: (1) mutation analysis is performed on an existing, established test suite; (2) developers then focus on surviving mutants that are not already detected by the current tests; and (3) test improvements are targeted at individual surviving mutants and applied incrementally [55, 60, 66].

In practice, step (3) is inherently constrained by limited budgets: only a subset of surviving mutants can be surfaced, selected, and practically resolved with targeted improvement in an iteration [55]. These constraints arise from several factors: (a) the high computational cost of mutation analysis [49], which has motivated extensive work on mutant reduction, subsumption, and prioritization [10, 27, 28, 36, 38, 42, 70] to “run fewer, faster, and smarter”, as summarized by Offutt and Untch [49]; (b) the recognition that only a subset of mutants is practically useful for inducing meaningful test improvements, captured by notions such as equivalent, stubborn, productive, and actionable mutants [8, 55, 69]. In practice, at Google, only 38% of surfaced mutants are resolved, rising to only 60% even when the mutants are explicitly endorsed by human reviewers [55]; (c) the substantial manual effort required of developers, as consistently reported in human studies [52, 55, 71]; and (d) the reality that mutation adequacy goals are rarely pursued to 100% in practice.

Moreover, mutant killing is inherently open-ended: for a given surviving mutant, multiple valid killing strategies may exist, and different developers may arrive at different solutions. Prior work shows that, in practice, targeting a specific surviving mutant can lead to diverse solutions, including edits to production code, existing test cases, or adding entirely new tests, as reported in Google’s industrial study [55] and empirical studies of mutant killing causes [17].

Limitations of Existing Evidence. Despite decades of empirical research and the theoretical intuition underlying the coupling-effect hypothesis—namely, that “*in many cases tests of a program that uncover simple errors are also effective in uncovering much more complex errors*” [15], we still

surprisingly lack a clear understanding of whether, how, and to what extent the mutant killing action meaningfully contributes to detecting or preventing future real bugs. This gap stems from the unique, structured, and uncertain nature of mutant-killing activities and from the fundamental evaluation asymmetries between mutant killing and real-bug detection.

In particular, surveys [60], experience reports [3], and case studies [14, 66] in mutation testing provide rich, contextualized insights into real-world deployments. However, these studies are observational and therefore limited in their ability to isolate and quantify the causal impact of killing surviving mutants on the prevention of future, previously unseen real bugs—that is, the actual value of the mutant-killing tests that developers might write.

Complementary to these limitations, much prior research adopts a retrospective perspective, starting from known real bugs and their associated bug-fixing tests to reason about whether mutation testing would induce similar tests. A common approach is to analyze whether surviving mutants are coupled to real bugs—either via bug-fixing (fault-triggering) tests [15, 35] or through similarities in test-execution behavior or semantics [25]. For example, Just *et al.* [35] examine whether triggering tests (usually bug-fix tests [56]) that detect real faults also kill additional, otherwise surviving mutants, and interpret such mutants as being coupled. A similar retrospective reasoning appears in Google’s study [54], which investigates whether mutation testing would have surfaced surviving mutants that were subsequently killed post hoc by bug-fixing tests.

Crucially, these analyses rest on the assumption that killing a coupled mutant would induce the same or an equivalent bug-fixing test, which implicitly treats the notion of coupling as a test-contingent artifact that holds only under specific triggering tests. This assumption is tenuous for two reasons. First, mutant killing is inherently open-ended: different developers may write different tests to kill the same surviving mutant, resulting in substantially different test designs. Second, bug-fixing tests are typically written with explicit knowledge of the real bug [56], rather than being derived from mutant-killing activities. As a result, it is unclear whether developers would have written the same bug-fixing tests through mutant killing—and, if not, whether alternative mutant-killing tests would have detected the real bug. Consequently, without evidence that such coupling is robust across alternative mutant-killing tests, we cannot confidently conclude that mutant-killing activities are likely to translate into reliable real-bug detection or prevention.

To make this a bit more clear, consider that to demonstrate the coupling effect for a real bug and its bug-triggering test, one need only establish the existence of *any* surviving mutant that would have been killed by that test. This is the empirical approach adopted by existing studies [35, 56]. Although these studies provide the critical foundation of demonstrating that the coupling effect truly exists, this is not the same as showing that a specific test that is designed to kill a surviving mutant (without any knowledge of real bugs that actually occurred later) would also detect actual future real bugs. There are often many test cases that can kill a single surviving mutant—some of these may detect future real bugs, and some may not. In sum, although existing studies demonstrate the existence of the coupling effect, the research does not yet show the extent to which killing surviving mutants (without knowledge of future real bugs) helps to detect future real bugs.

Challenges of Directly Measuring Mutant-killing Effectiveness. In principle, one could study the effectiveness of mutant killing by starting from a known real bug documented by a bug-fixing (triggering) test. Developers would then perform mutation analysis on the corresponding test suite without knowledge of the bug (*e.g.*, removing the bug-fixing test). Their task would be to identify and kill surviving mutants by writing new tests. Finally, one could observe whether the resulting mutant-killing tests would also expose that specific real bug.

However, conducting such a study at scale with human participants is prohibitively expensive and would remain subject to substantial uncertainty due to developer-specific choices and variability in test design. Moreover, from an industrial perspective, it is neither practical nor motivating for

developers or companies to improve a test suite for a fault that has already been identified and fixed, especially when the improvement effort provides no immediate benefit to production systems.

To address this challenge, we conduct the first large-scale controlled experiment to investigate whether, how, and how likely mutant killing helps prevent or detect future real bugs. This experiment is enabled by a targeted, automated mutant-killing technique introduced in the next section.

3 Targeted Mutation Killing Technique

This section presents our targeted mutant-killing technique and its role in enabling controlled experimentation. We begin with a motivating example and then describe a fully automated approach that performs attributable analysis to generate targeted test improvements via assertion augmentation. We position the technique within a taxonomy of fault-detection gaps grounded in a fault error propagation model [41] and explain how its design serves our experimental objectives.

3.1 Motivating Example

Figure 1 shows a real-world test case from APACHE CLI version 1.2. This version contains a bug reported in a Jira report [1] and later included in the DEFECTS4J real-bug dataset as CLI-31 [35]. In this version, none of the existing tests detect the bug, including the one shown in the figure.

An augmentation that could have detected this bug is adding the assertion highlighted in green, which validates that `e.option.argName` is null. This assertion detects the real bug by exposing its effect of changing `argName` to a non-null value. We refer to this operation as *assertion augmentation*, i.e., the addition of assertions to existing test cases. This approach can be categorized as a type of test amplification [12], i.e., reusing or transforming existing tests to synthesize new tests.

In principle, this additional assertion could be derived by comparing the runtime values of `e` when executing the test case on the fixed and buggy program versions. However, this discrepancy is only observable with prior knowledge of the real bug, which is unavailable during test development in practice. This leads to the question: can mutations act as proxies for real bugs, enabling targeted augmentation that improves a test suite's ability to detect or prevent unseen future real bugs? To explore this question, we introduce a fully automated assertion-augmentation technique.

3.2 Mutation-Based Assertion Augmentation

Figure 2 presents the workflow of our technique. Given a test suite, an original program, and a faulty program (e.g., a mutant), the technique produces a set of mutant-killing assertion augmentations. Each augmentation adds a single validated assertion to an existing test case, yielding an augmented test case that kills the mutant. The technique comprises three stages: (1) state collection, (2) augmentation-specification generation, and (3) test transformation and execution validation.

3.2.1 State Collection. The technique begins by instrumenting test code to systematically collect values of *output variables*, i.e., variables that are accessible within the test fixture or can be asserted on. These include return values of invoked program units, values read from objects reachable via the test fixture, and other program variables accessible by the test during execution. For example, in Figure 1, such variables include `args` (line 1), the return value of `parse` (line 3), and the exception

```
public void testTwoLongOptionsFromGroup() throws Exception {
1  String[] args = new String[] { "--file", "--directory" };
2  try {
3      parser.parse(_options, args);
4      fail("two arguments from group not allowed");
5  } catch (AlreadySelectedException e) {
6      assertNull(e.option.argName);
7      assertNotNull("null option group", e.getOptionGroup());
8      assertEquals("selected option", "f", e.getOptionGroup().getSelected());
9      assertEquals("option", "d", e.getOption().getOpt());
10 }
11 }
```

Added assertion detecting the real bug

Fig. 1. Assertion augmentation example

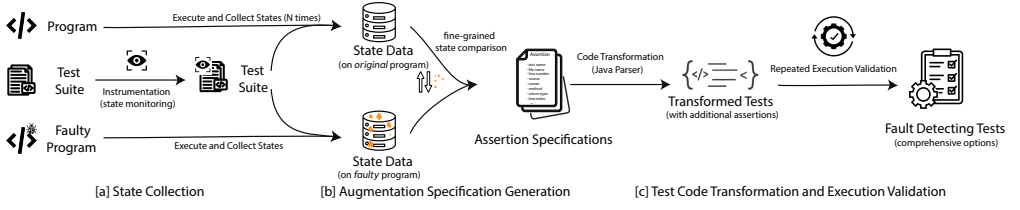


Fig. 2. Fault-based Assertion Generator

object `e` (line 5). Output variables also include intermediate expressions from chained calls (e.g., the return value of `getOption` in line 9).

Next, the technique executes the instrumented tests on both the original program (repeated multiple times) and the faulty program, collecting runtime states of all output variables. For object variables, it recursively traverses reachable fields up to a bounded depth, which determines how much of the object state is observed, and records the extracted values in a structured representation that preserves their hierarchical access paths. For example, Figure 3(a) illustrates the collected state for the variable `book`, which contains multiple fields (e.g., `title`, `extra`) that may themselves contain nested fields. Each reachable field constitutes a potential assertion target. These design choices correspond to the state-observation scope, state-extraction depth, and state-extraction approach three dimensions studied in our prior work [20].

3.2.2 Augmentation Specification Generation. Next, we compare the collected states from executions on the original and faulty programs to identify state differences introduced by the fault. To filter nondeterministic behavior, we exclude any field whose value varies across repeated executions.

State comparison is performed via depth-first traversal of the object graphs to localize differences at the finest possible granularity: when feasible, discrepancies are identified at the deepest access paths from an output variable, such as `book.title` or `book.author.tags[0]` in Figure 3(b). However, the fault may sometimes alter the structure of the output state. For example, Figure 3(c) shows `book.author.tags` becoming empty or `null`. Figure 3(d) shows a dynamic type change in `book.extra`. In extreme cases the root output variable `book` becomes `null`. The technique detects such structural mismatches, including changes in nullability, dynamic type, or collection size, and terminates deeper comparisons along the affected branch.

This produces *assertion-augmentation specifications*, which record (1) the variable location (i.e., the assertion site), (2) the access path to be asserted (e.g., `book.title`), and (3) the expected value observed during execution of the original program. The technique generates multiple specifications when multiple state differences are observed, each corresponding to a fault-detecting assertion.

3.2.3 Test Code Transformation and Execution Validation. For each specification, the technique applies a localized code rewrite to the corresponding test case, producing an augmented variant with a single additional assertion. The transformation is performed on the test’s abstract syntax tree (AST) and consists of three steps: (1) declaring a fresh local variable, when needed, (2) assigning the target expression to the variable, and (3) inserting an assertion that checks an attribute from the specified access path against the expected value. This transformation preserves the original test’s control flow, inputs, and execution context, and introduces only a localized edit. Each generated assertion is then validated independently by execution: it must pass on the original program and fail on the faulty program. To mitigate flaky behavior, validation is repeated multiple times, and only assertion augmentations that consistently satisfy these criteria are accepted.

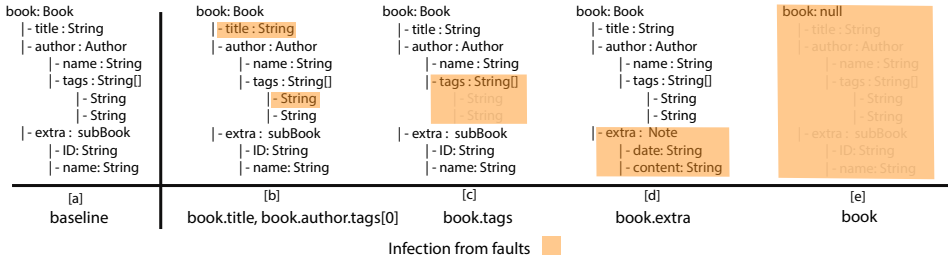


Fig. 3. Fault-induced deviations on an assertable program variable. Panel [a] shows the baseline state of *book*, while Panels [b]–[e] illustrate erroneous states with colored regions marking fault-induced state infections.

Technique Characteristics Supporting Controlled Experiments. The proposed technique supports controlled experiments on individual mutant killing activities through four key characteristics: (1) it introduces only localized edits to existing tests; (2) each augmentation is directly attributable to a specific mutant; (3) it produces a finite set of alternative mutant-killing assertions that can be examined systematically; and (4) it can operate on general code changes, enabling the same assertion-augmentation process to be applied directly to real bugs beyond mutations, as illustrated in Figure 2. This enables a controlled comparison between mutant-derived and real-fault-derived assertions without introducing additional mechanisms or assumptions.

3.3 Positioning the Technique via a Taxonomy of Fault-Detection Gaps

To contextualize the scope of the technique, we adopt the fault–error propagation (RIPR) model [18, 41] to characterize the necessary conditions for a test to detect a fault. Under the RIPR model, a test must (i) execute the faulty code (*reachability*), (ii) produce an erroneous program state (*infection*), (iii) propagate that erroneous state to an observable output (*propagation*), and (iv) reveal the failure via test oracles (e.g., assertions; *revealability*).

When a test suite fails to detect a fault (e.g., a surviving mutant), the RIPR model helps characterize the underlying weakness in terms of missing fault-detection conditions. For our analysis, we organize these conditions into a taxonomy of three categories: (1) *reachability gaps*, where no test executes the faulty code; (2) *infection-propagation gaps*, where an erroneous state is not produced or does not propagate to an observable point; and (3) *revealability gaps*, where an observable erroneous state exists but is not verified by test oracles (e.g., assertions).

The proposed taxonomy categorizes the minimal actions needed to improve a test suite to detect a specific undetected fault: addressing reachability and infection-propagation gaps typically requires generating new inputs or execution scenarios. This is an open-ended space that has been extensively studied in search-based testing [59, 65]. In contrast, addressing revealability gaps requires strengthening test assertions to expose erroneous program outputs observable in test code, as in our technique. Specifically, assertion augmentation aligns with a developer-reported mutant-killing use case, where some practitioners report that they kill surviving mutants to confirm that “the code and tests are making the strongest assertions they can” [60].

Terminology and scope. For our convenience, unless otherwise specified, throughout the remainder of this paper, the terms *killable*, *detectable* are defined relative to the assertion-augmentation technique used in our controlled experiment. Please note that other mutant-killing approaches may also kill surviving mutants, including by addressing reachability or infection-propagation gaps, but such approaches are outside the scope of our analysis. We discuss the implications of this scope limitation further in the threats to validity section.

4 Experimental Design and Research Questions

The technique introduced in Section 3 provides a concrete, automated means of killing individual mutants and integrating the resulting augmentations into the test suite. This capability enables us to operationalize the hypothetical experiment described at the end of Section 2: we perform mutation analysis and targeted mutant killing on a test suite that fails to detect a specific bug, and then evaluate whether the resulting augmentations would detect it.

Accordingly, in this section, we introduce a controlled experimental framework that models mutant-killing activities and evaluates their impact on real-bug detection using the Defects4J real-bug dataset [34]. We then enumerate the research questions that guide our investigation.

4.1 Experimental Design and Notations

Our experimental design decomposes the mutation-based test strengthening (mutant-killing process) and evaluation process into a sequence of controlled stages; this enables us to systematically analyze the multifaceted details of how mutant killing translates into real-bug detection.

(i) Defects4J dataset. DEFECTS4J is a real-bug dataset for Java, curated by Just *et al.* [34]. For each DEFECTS4J bug, we consider a buggy program version V_b and its corresponding fixed version V_f , which differ by a single bug-fixing patch. Associated with each version pair, DEFECTS4J includes a developer-written test suite, denoted T^+ , runnable on both versions. By construction, T^+ contains one or more triggering tests $\hat{t}$ that fail on V_b and pass on V_f , as indicated by the ‘+’ superscript.

(ii) Bug-Ignorant Test Suites. We construct a *bug-ignorant* test suite T by removing all triggering tests $\hat{t}$ from T^+ . The resulting suite is therefore unaware of the bug and passes when executed on V_b . We use T as the starting point for mutation analysis and mutant-killing activities.

(iii) Mutation Analysis and Surviving Mutants. We generate a set of mutants $\mathcal{M}$ on program V_f and perform mutation analysis on T by running the test suite on each mutant. Specifically, executing the mutants in $\mathcal{M}$ against T identifies the subset of surviving mutants, $\mathcal{M}_{\text{surv}}(T) \subseteq \mathcal{M}$.

(iv) Mutant Killing Opportunities. We apply our mutant-killing technique to each surviving mutant $m \in \mathcal{M}_{\text{surv}}(T)$ to generate mutant-killing options. If m is killable, the technique produces a set of assertions $\mathcal{A}_{\text{mut}}(m)$, each of which can be added to T to kill m . We denote the subset of killable mutants by $\mathcal{M}_{\text{surv}}(T)^+ \subseteq \mathcal{M}_{\text{surv}}(T)$, where the superscript ‘+’ indicates that these mutants can contribute test augmentations (*i.e.*, additional assertions). As defined in Section 3.3, *killable* refers only to mutants that can be killed using our assertion-augmentation technique, rather than through other interventions, such as adding new test inputs or fixtures.

(iv) Mutant Killing Modeling under Practical Constraints. Motivated by Section 2, we model mutant killing as an *incremental, selective* process under limited budgets.

Select one mutant-killing option. For a given killable surviving mutant $m \in \mathcal{M}_{\text{surv}}(T)^+$, multiple generated assertions may be capable of killing it, forming the set $\mathcal{A}_{\text{mut}}(m)$. Since any single assertion in this set is sufficient to kill m , we model a mutant-killing solution by randomly selecting one assertion $a^* \in \mathcal{A}_{\text{mut}}(m)$. The resulting augmented test suite is denoted $T^\uparrow \triangleq T \cup \{a^*\}$.

Resolve only a subset of mutants. Second, only a subset of killable mutants is randomly selected for killing, which we model via a mutant-killing intensity $\alpha \in (0\%, 100\%]$. Such a subset is denoted $S = \mathcal{M}_{\text{surv}}(T)_\alpha^+ \subseteq \mathcal{M}_{\text{surv}}(T)^+ = \{m_1, \dots, m_n\}$, with $k = |S| = \alpha \cdot |\mathcal{M}_{\text{surv}}(T)^+|$ denoting the number of selected mutants to be killed. This yields the strengthened test suite

$$T_\alpha^\uparrow \triangleq T \cup \bigcup_{m \in \mathcal{M}_{\text{surv}}(T)_\alpha^+} \{a_m^*\}.$$

We then evaluate whether the strengthened test suite $T_\alpha^\dagger$ detects the real bug. Because mutant killing is modeled as selective, we use a probabilistic model to model the real bug detection probability based on observable outcomes. Specifically, we assume that (i) for each killable surviving mutant m_j , one mutant-killing assertion is chosen uniformly at random from the finite set of available options $\mathcal{A}_{\text{mut}}(m_j)$; (ii) a subset of killable surviving mutants S is selected at random according to a given mutant-killing intensity α ; and (iii) each mutant is killed independently. Under these assumptions, all probabilities in our analysis are computed directly from exhaustive measurements over finite mutant-killing assertion sets.

This model is useful in two ways: because every mutant-killing assertion option is enumerated, real-bug detection probabilities can be computed exactly from exhaustive measurements rather than estimated through sampled simulations; and because it assumes no particular mutant-selection strategy, it provides a strategy-agnostic *baseline* for expected real-bug detection under a given mutant-killing budget. However, these probabilities remain contingent on the randomness and independence assumptions above, which abstract away practitioners' actual mutant prioritization and assertion-writing behavior; we revisit this limitation when interpreting the results in Section 8.

Specifically, for a killable surviving mutant m_j , $\mathcal{A}_{\text{mut}}^R(m_j) \subseteq \mathcal{A}_{\text{mut}}(m_j)$ denotes the assertion augmentations that also detect the real bug b . The probability that a *randomly* selected assertion for m_j fails to detect b can be viewed as a Bernoulli trial [58] with a mutant-specific probability:

$$q_{m_j} \triangleq 1 - \frac{|\mathcal{A}_{\text{mut}}^R(m_j)|}{|\mathcal{A}_{\text{mut}}(m_j)|},$$

Assuming independence across mutants, killing k randomly selected mutants from the n killable surviving mutants $\mathcal{M}_{\text{surv}}(T)^+$ can be viewed as k independent Bernoulli trials for detecting b , with the expected failure probability given by

$$\mathbb{E}[\text{fail} \mid k] = \frac{1}{\binom{n}{k}} \sum_{\substack{S \subseteq \mathcal{M}_{\text{surv}}(T)^+ \\ |S|=k}} \prod_{m \in S} q_m, \quad (1)$$

where we iterate over all mutant selecting options $\binom{n}{k}$ (the number of ways to choose k from n) and average the failure detection probability ($\prod_{m \in S} q_m$) across all size- k selections of killable mutants. The corresponding detection probability for a specific real bug is $P_{\text{detect}}(T_\alpha^\dagger) = 1 - \mathbb{E}[\text{fail} \mid k]$.

(v) Additional Analysis. While our technique is designed to kill mutants, we also consider a setting in which a mutant is substituted by the real bug b . We then apply the same technique to T and denote the resulting set of *real-bug-derived* assertions by $\mathcal{A}_{\text{real}}(b)$.

This additional analysis serves three purposes: (1) it establishes an upper bound on the potential of the specific mutant-killing approach we employ. Specifically, it identifies real bugs that are, in principle, detectable by augmenting T with additional assertions. This mitigates the *revealability gaps* described in Section 3.3; (2) it enables a direct comparison between assertions generated by targeting mutants and those generated by targeting real bugs, namely $\mathcal{A}_{\text{mut}}(m)$ versus $\mathcal{A}_{\text{real}}(b)$; and (3) it provides insight into when and why mutation-based strengthening succeeds or fails at detecting real bugs, as reflected in $\mathcal{A}_{\text{mut}}^R(m)$ detecting b in contrast to $\mathcal{A}_{\text{real}}(b)$.

4.2 Research Questions

Following the experimental design in Section 4.1, we introduce a structured set of research questions that progressively narrow the analysis, beginning with an understanding of the dataset and advancing to investigate the multifaceted details of mutant killing, including available options, effectiveness, underlying mechanisms, and practical impact.

RQ1: How many real bugs do the test suites fail to detect due to reachability, infection-propagation, or revealability gaps? We first investigate which real bugs are, in principle, detected through assertion augmentation applied to existing tests. Specifically, following the taxonomy introduced in Section 3.3, we categorize test-suite weaknesses in terms of reachability gaps, infection-propagation gaps, and revealability gaps with respect to a corresponding bug. For bugs exhibiting revealability gaps, our technique can successfully generate additional assertions that enable their detection. This analysis characterizes the dataset and establishes the scope of mutation-based assertion augmentation in our subsequent controlled experiments.

RQ2: How many surviving mutants can be killed through our technique, and how many assertions does it generate? We next perform mutation analysis on the programs identified in RQ1 and apply our mutant-killing technique to surviving mutants. We study the potential outcomes of targeting each surviving mutant individually using our technique. Specifically, we quantify (1) how many surviving mutants are surfaced from mutation analysis, (2) how many of these mutants are killable via assertion augmentation, and (3) how many mutant-killing options, *i.e.*, assertions, can be generated overall. The results characterize the complete, finite space of mutant-killing solutions that can be applied through our technique under the controlled experimental setting.

RQ3: Do assertions introduced to kill surviving mutants also detect real bugs? Building on the mutant-killing options identified in RQ2, we examine whether these solutions also detect real bugs. First, we quantify how many real bugs can be detected via mutant killing; if not, we identify where the mutation-based workflow fails, *i.e.*, when (a) no relevant surviving mutants are identified, (b) identified surviving mutants yield no assertions under our technique, or (c) generated mutant-killing assertions fail to expose the real bug. Second, for detectable real bugs, we quantify the number of mutants that can contribute to real-bug detection, *i.e.*, those that are considered “coupled” to real bugs through triggering tests by prior work [35]. Third, we quantify how many mutation-derived assertions are capable of detecting real bugs.

RQ4: How likely is a prescribed mutant-killing intensity to detect real bugs under realistic testing effort? Because applying all solutions identified in RQ2 to T is unrealistic in practical mutant-killing scenarios (as discussed in Section 2), RQ4 estimates the probability that the strengthened test suite $T_\alpha^\uparrow$ detects a real bug under varying mutant-killing intensities α (defined in Section 4.1). In the modeled mutant-killing scenario, only a subset of surviving mutants is selected, and resolving each selected mutant requires adding only a single assertion. Specifically, for each real bug, we estimate its detection probability under mutant-killing intensities $\alpha \in \{5\%, 10\%, 20\%, \dots, 100\%\}$. These probabilities are computed using Formula (1) in Section 4.1, which relies on the probabilistic modeling and its randomness and independence assumptions.

RQ5: How do mutant-derived assertions differ from real-bug-derived assertions, and what do mutant-derived assertions look like when they detect real bugs? To assess the validity of using mutants as proxies for real bugs, we substitute coupled mutants with real bugs and compare their augmentation outcomes. Specifically, we compare the *scale* of generated assertions by examining the numbers of augmentable tests, assertable variables, and distinct assertion options, as well as assertion specificity, measured by access-path depth (*e.g.*, `book.author.tags`, depth 3). These measures characterize the generated assertions across different fault sources and also reflect the scale of the corresponding fault effects. Finally, we contrast mutant-derived assertions that also detect real bugs, $\mathcal{A}_{\text{mut}}^R(m)$, with assertions derived directly from real bugs, $\mathcal{A}_{\text{real}}(b)$, to examine whether they target identical program states and exhibit similar structures.

5 Experimental Setup

This section describes the experimental setup for evaluating mutant-killing effectiveness on real bugs using the technique, along with optimizations that make the analysis tractable at scale.

Defects4J Dataset. We base our experiments on DEFECTS4J version 3.0.1, which contains 854 real bugs across 17 subjects. We exclude two subjects. MOCKITO is excluded due to known unreliability of its mutation analysis results, as documented in an open issue [33] at the time of our submission. CLOSURE is excluded because most of its tests rely on centralized AST-equivalence oracles rather than localized behavioral assertions, which limits the applicability of assertion augmentation in our setting. After these exclusions, our analysis covers 642 real-bug versions.

Mutation Analysis. We use Major [37] integrated with DEFECTS4J. Major provides a set of mutation operators that introduce localized syntactic changes consistent with simple-mutation assumptions underlying the coupling effect hypothesis. We apply all eight operators, restricting mutation generation and test execution to the relevant classes identified by DEFECTS4J.

Fault-based Assertion Generation. We extend DEFECTS4J with additional commands to support our fault-based assertion generation pipeline, which operates uniformly on either real bugs or surviving mutants. Specifically, to collect program states, we perform bytecode instrumentation using ASM [9] library on the test suite. Collected states are stored in a structured JSON representation that preserves object graph relationships to support state comparison. This is achieved by recursively traversing fields up to a fixed depth of five while avoiding redundant serialization.

Code generation is performed by rewriting test cases at the abstract syntax tree level according to generated assertion specifications (see Section 3) using the JavaParser [2] library. Each accepted generated assertion undergoes at least seven executions in total: two baseline runs on the original program version, one run on the faulty version for state collection, two validation runs that must pass on the original program, and two validation runs that must fail on the faulty program.

Operational Optimizations. To make computationally expensive mutation testing tractable, we apply two optimizations. First, we perform *selective mutation analysis* by first attempting to generate assertions directly from real bugs. If no real-bug-derived assertion is generated, the bug necessarily exhibits either a *reachability gap* or an *infection-propagation gap*, rendering assertion augmentation ineffective in principle; such cases are reported in RQ1. For such program versions, we skip mutation analysis entirely. Second, we apply *coverage-guided test selection*. Specifically, we perform a fine-grained coverage analysis to identify test methods that execute bug-relevant behavior, supporting reachability classification while reducing assertion generation cost.

To support coverage analysis for real bugs, we identify *bug-impacted lines* from diff hunks in the patch. A test method is considered to exercise a real bug if, in either version, it (1) executes a bug-impacted line, (2) instantiates a bug-impacted class when its instance fields change, or (3) loads a bug-impacted class when static fields change or when classes are added or removed.

Additionally, we use our technique to target *relevant surviving mutants*, i.e., surviving mutants covered by tests that are augmentable when targeting the real bug. While killing other surviving mutants may be useful in practice, in our experiment, such augmentation cannot contribute to real-bug detection and would therefore waste effort relative to our evaluation goal.

Experimental Cost. Due to the substantial computational demands, we ran the experiments on three machines: a Mac mini (M4), a MacBook Pro (2021, Apple M1 Pro), and a Linux x86_64 system (Intel Core i7-950). The analysis required approximately 1,177 hours in total, including 266 hours for real-bug assertion generation, 435 hours for mutation analysis, 430 hours for surviving-mutant assertion generation, and 47 hours for validation on real bugs. This experimental cost reflects the overhead of mutation analysis and exhaustive enumeration of validated mutant-killing options, as

Table 1. Fault-detecting Gaps for Test Suites (T) to Detect Real Bugs from DEFECTS4J

Subject	# Bugs (from DEFECTS4J)				# Assertions
	All Bugs	Reach Gap: Bugs not executed by T	Infect-Prop. Gap: Bugs not creating detectable infections	Reveal Gap: Bugs w/ detectable infections; but not detected by T	(Generated for Bugs in Reveal Gap)
JACKSONDATABIND	110	14	76	20	4,003
MATH	106	21	68	17	974
JSOUP	93	8	73	12	4,878
LANG	61	11	47	3	129
COMPRESS	47	0	38	9	64
CLI	39	9	19	11	1,868
CHART	26	6	14	6	150
TIME	26	0	20	6	709
JACKSONCORE	26	6	12	8	244
COLLECTIONS	28	6	20	2	74
JxPATH	22	3	18	1	6
GSON	18	2	13	3	33
CODEC	18	0	15	3	105
Csv	16	1	13	2	13
JACKSONXML	6	3	2	1	16
Total	642	90	448	104	13,266

well as evaluations on real bugs, rather than the cost of applying the technique in practice. Detailed per-bug and per-stage time costs are reported in the accompanying artifact.

6 Results

RQ1: How many real bugs do test suites fail to detect due to reachability, infection-propagation, or revealability gaps?

Table 1 summarizes, for each subject, which real-bug detection conditions are not satisfied by the test suite T . Specifically, it reports the number of real bugs along with a stage-wise breakdown of missing detection conditions based on the taxonomy in Section 3.3. The last column reports the number of generated assertions that enable the detection of real bugs exposing revealability gaps.

For example, in JACKSONDATABIND, 14 of 110 bugs expose reachability gaps, as no test in T executes the buggy code. Another 76 bugs expose infection-propagation gaps: although the buggy code is executed, either the program state is not infected or the infected state does not propagate to observable test outputs. The remaining 20 bugs expose revealability gaps, where additional assertions suffice for detection. For these bugs, our technique generates 4,003 distinct assertions, each capable of turning an existing passing test into a failing (triggering) one.

In total, 90 bugs (14.0%) uncover reachability gaps, 448 (69.8%) uncover infection-propagation gaps, and 104 (16.2%) uncover revealability gaps. Bugs in the first two categories generally require introducing new test inputs or execution scenarios to enable detection. Bugs exposing revealability gaps can be detected by strengthening existing tests with additional assertions, without requiring new test inputs or execution scenarios. For this category, our technique generates 13,266 assertions derived directly from these bugs, each of which can independently detect the bug.

Scope of Subsequent Analysis. Our mutation analysis and mutant-killing experiments focus on the 104 real bugs associated with test suites with revealability gaps.

Table 2. Surviving Mutant-killing Outcomes and Real-bug Detectability

Subject	Mutant Killing			Real-Bug Detectability		
	#Surv. M.	#Killable M.(%)	#Gen. Asserts.	#Coupled M. (%)	#Detecting Asserts. (%)	
JACKSONDATABIND	1384	666 (48.12%)	5503	263 (39.49%)	4069 (73.94%)	
MATH	2274	888 (39.05%)	780	194 (21.85%)	185 (23.72%)	
JSOUP	2958	283 (9.57%)	11837	205 (72.44%)	1406 (11.88%)	
LANG	212	2 (0.94%)	4	2 (100.00%)	2 (50.00%)	
COMPRESS	1139	255 (22.39%)	469	120 (47.06%)	46 (9.81%)	
CLI	372	108 (29.03%)	782	77 (71.30%)	627 (80.18%)	
CHART	248	31 (12.50%)	109	12 (38.71%)	39 (35.78%)	
TIME	356	212 (59.55%)	689	106 (50.00%)	178 (25.83%)	
JACKSONCORE	1990	860 (43.22%)	1882	121 (14.07%)	222 (11.80%)	
COLLECTIONS	9	0 (0.00%)	0	—	—	
JXPATH	42	0 (0.00%)	0	—	—	
GSON	14	4 (28.57%)	5	4 (100.00%)	4 (80.00%)	
CODEC	772	419 (54.27%)	196	202 (48.21%)	113 (57.65%)	
CSV	22	1 (4.55%)	2	1 (100.00%)	2 (100.00%)	
JACKSONXML	40	13 (32.50%)	13	12 (92.31%)	6 (46.15%)	
Total	11832	3742 (31.63%)	22271	1319 (35.25%)	6899 (30.98%)	

RQ2: How many surviving mutants can be killed through our technique, and how many assertions does it generate?

The left portion of Table 2 reports, for each subject, the number of relevant surviving mutants identified through mutation analysis, the proportion killable by our technique, and the total number of generated assertions that independently kill them. For example, in JACKSONDATABIND, mutation analysis identified 1,384 relevant surviving mutants across 20 program versions (in Table 1). Of these, 666 (48.12%) are killable, resulting in 5,503 mutant-killing assertions.

The killability of surviving mutants and the availability of corresponding killing assertions vary across projects. An extreme case is JSOUP, where only 283 of 2,958 mutants (9.57%) are killable, yet this small subset yields a large number of assertions (11,837), reflecting substantial diversity in mutant-killing opportunities. No relevant surviving mutants are killable in COLLECTIONS or JXPATH. Both projects contain very few program versions with test suites exhibiting revealability gaps (Table 1) and only a limited number of surviving mutants (Table 2).

Surviving Mutant Killability Overall, across all experiments, 3,742 of 11,832 relevant surviving mutants (31.63%) are killable through our technique, yielding 22,271 mutant-killing assertions.

RQ3: Do assertions introduced to kill surviving mutants also detect real bugs?

We next examine if killing mutants using our technique helps detect real bugs. Figure 4 summarizes real-bug detectability through mutant killing across subjects. The stacked segments indicate if killing mutants can detect real bugs and, if not, the stage at which the mutant-killing pipeline fails.

For example, in JACKSONDATABIND, mutant killing detects real bugs in 14 of 20 instances. The remaining six instances fail at different stages of the pipeline: (1) in three cases, mutation

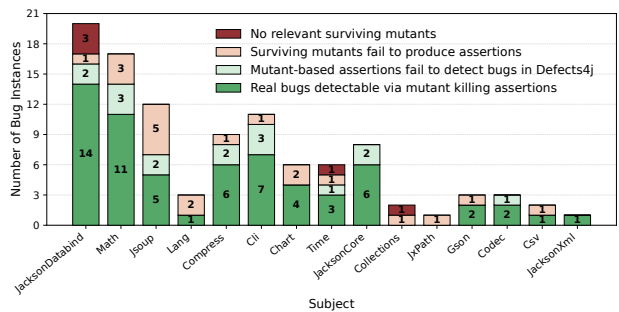


Fig. 4. Breakdown of real-bug detectability and non-detection reasons.

analysis yields no relevant surviving mutants: all mutants relevant to the bug are already killed by the existing test suite, *i.e.*, no mutants survive; (2) in one case, surviving mutants exist but do not produce mutant-killing assertions because these mutants produce no observable state infection in the test code; and (3) in two cases, mutant-killing assertions are generated but fail to detect the corresponding real bugs in the dataset. For the last case (shown in light-green), although assertions derived from mutants cannot detect the bugs in DEFECTS4J, they may still help prevent future real bugs not present in the dataset.

Real-bug Detectability. Mutation-based improvement can detect 63 of 104 real bugs (60.6%); failures occur at pipeline stages: no surviving mutants (5, 4.8%), missing killable mutants (20, 19.2%), or ineffective test augmentations to detect real bugs in the dataset (16, 15.4%).

Next, we assess how frequently mutant-killing opportunities translate into real-bug detection. Specifically, we evaluate the fraction of killable surviving mutants that are “coupled” to real bugs—*i.e.*, surviving mutants whose augmentations can detect real bugs—and the fraction of generated assertions that do so. The right portion of Table 2 reports these results. For example, in JACKSONDATABIND, 263 of 666 killable surviving mutants (39.49%) yield at least one assertion that detects real bugs (*i.e.*, 263 coupled mutants). At the assertion level, 4,069 of 5,503 mutant-killing assertions (73.94%) detect real bugs.

Real-bug Detection Opportunities. Overall, 1,319 of 3,742 killable surviving mutants (35.25%) are coupled to real bugs through at least one mutant-killing assertion that detects a real bug. Each of the 63 detectable real bugs is coupled to 21 surviving mutants on average (median 9; maximum 172). Of 22,271 mutant-killing assertions, 6,899 (30.98%) detect real bugs.

RQ4: How likely is a prescribed mutant-killing intensity to detect real bugs under realistic testing effort?

Although our analysis shows that 63 real bugs are detectable through mutant-guided assertion augmentation in our experiment, this potential may not translate into effective real-bug detection under realistic mutant-killing effort. We simulate a plausible mutant-killing workflow in which a random $\alpha\%$ subset of killable surviving mutants is selected; for each selected mutant, we randomly choose one mutant-killing assertion. The resulting probability that the augmented test suite detects a real bug is denoted $P_{\text{detect}}(T_{\alpha}^{\uparrow})$ (defined and computed in Section 4.1).

To help interpret the data, we first demonstrate the outcome of killing 5% of the killable surviving mutants ($\alpha = 5\%$). For each of the 63 real bugs, we compute a detection probability $P_{\text{detect}}(T_{\alpha}^{\uparrow})$ from 0% to 100%. Figure 5 (left) presents a mirrored bar chart showing how individual detection probabilities are distributed across different ranges. Under this setting, 2 of the 63 real bugs have detection probabilities below 10%, whereas 19 fall within the 90–100% range. This means these 19 real bugs can be easily detected when randomly killing only 5% of the killable mutants. The centered bar chart layout highlights the strong skew toward high detectability at low killing intensity.

Figure 5 (right) uncovers the full distribution of individual real-bug detection probabilities using violin plots [30] across different mutant-killing intensities α . The width of each violin indicates how frequently detection probabilities occur at each level, providing an intuitive view of their density. We additionally plot individual per-bug detection probabilities as black dots and overlay mean and median trends to aid interpretation.

When killing only 5% of killable surviving mutants (240 out of 22,271 assertions), the mean detection probability reaches 63.7%, indicating that even minimal mutant-killing effort can substantially improve real-bug detection. Increasing α from 5% to 30% raises mean detectability to 84.5%, after which returns diminish. This behavior reflects a multiple-opportunity effect: each

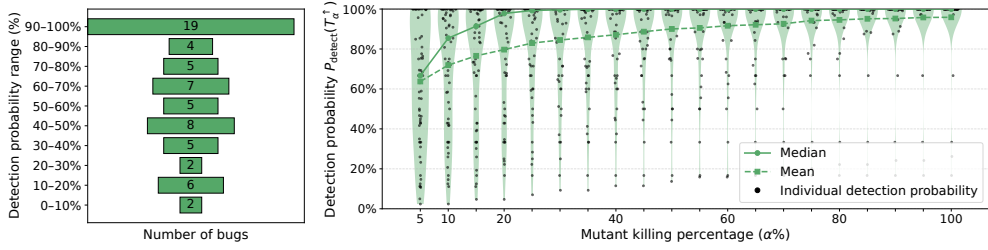


Fig. 5. (Left) Mirrored bar chart showing the number of real bugs in each detection-probability range when killing 5% of killable surviving mutants. (Right) Violin plot showing the distribution of per-bug detection probabilities across different mutant-killing intensities α .

coupled surviving mutant represents an independent Bernoulli-style trial with some probability of producing a fault-revealing assertion, and detection requires only one success. The more mutants selected to kill, the higher chances to detect the real bug. With an average of 21 coupled mutants per real bug ($RQ3$), even low-intensity mutant killing is likely to sample at least one coupled mutant, yielding a high detection probability. Overall, mean detection probabilities reach 87.1%, 90.1%, 94.6%, and 95.9% for $\alpha = 40\%$, 50%, 80%, and 100%, respectively. Notably, even when aiming to kill 100% of killable surviving mutants, detection probability still does not reach 100%, because only one assertion is selected per mutant—reflecting a realistic constraint where integrating all mutant-killing assertions would be impractical.

Detection Probability. Across real bugs that are detectable through mutant killing, high detection probability can be achieved with relatively low mutant-killing effort. Killing 30% of killable surviving mutants—using one randomly selected assertion per mutant—yields a mean detection probability of 84.5%. This is already close to the 95.9% achieved when killing all mutants.

RQ5: How do mutant-derived assertions differ from real-bug-derived assertions, and what explains their ability to detect real bugs?

Figure 6 presents grouped boxplots comparing assertion characteristics when directly targeting real bugs versus coupled mutants. Overall, assertions derived from real bugs are distributed substantially more widely in the test code: compared to mutation-derived assertions, they yield more assertion options (median: 11.5 vs. 2), inspect more variables (median: 6 vs. 2), and apply to more tests (median: 4 vs. 1). Also, mutation-derived assertions tend to inspect deeper access paths than real-bug-derived assertions (median: 3.8 vs. 2.4), producing finer-grained checks targeting more deeply reachable attributes.

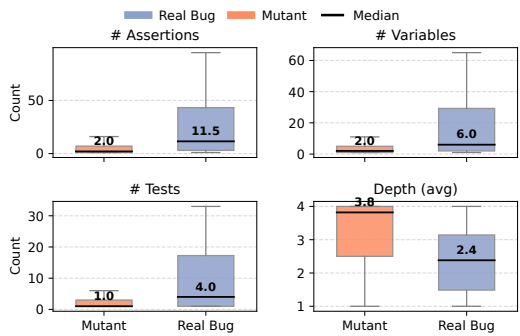


Fig. 6. Characteristics of assertions derived from real bugs and mutants.

We further examine mutation-derived assertions that successfully detect real bugs ($\mathcal{A}_{\text{mut}}^R(m)$). Among 6,899 such assertions, 2,798 (40.6%) are exactly identical to some generated by directly targeting the corresponding real bugs ($\mathcal{A}_{\text{real}}(b)$). This indicates that these assertions are derived from observing the same specific memory locations affected by the real bug. An additional 2,846

(41.3%) check program attributes whose access paths are strict subsets of those checked by real-bug-derived assertions. For example, an assertion on *book.author.tags* has an access path that is a strict subset of one checking *book.author*. Overall, at least 81.8% of mutant-derived assertions that detect real bugs validate program state that overlaps with, or is a subset of, the state impacted by the real bug.

Mutations vs. Real Bugs. Compared to mutation-derived assertions, real-bug-derived assertions are more broadly distributed across the test code and check shallower access paths that include more attributes. When mutation-derived assertions detect real bugs, they typically validate program outputs that overlap with, and are often strict subsets of, those affected by the real bugs.

7 Discussion and Implications

The experimental results from the previous section provide insights into how mutant killing contributes to real-bug detection. In our controlled experiment, we generated assertions guided by both mutants and real bugs, and compared the resulting effects of assertion augmentation. The ensuing comparative analysis sheds light on the practical mechanisms underlying the coupling effect between mutants and real bugs, and reveals the conditions under which mutant killing meaningfully improves or fails to improve test-suite fault-detection capability.

Effectiveness of Mutant Killing and Empirical Approximation of Coupled Mutants. Our results show that real bugs can be detected by killing the subset of mutants that survive due to revealability gap without the prior knowledge of the bug. Specifically, our assertion-augmentation technique detected 63 out of 104 detectable real bugs (60.5%) by killing surviving mutants.

Importantly, detecting these real bugs did not require exhaustively applying all mutant-killing assertions generated by our technique. In *RQ4*, we mimic the resource and time constraints developers face in practice: when killing a surviving mutant, we randomly select at most one generated assertion. Even under such stringent constraints, killing only 30% of the killable surviving mutants — each with a single new assertion — achieves an average real-bug detection probability of 84.5% for these 63 bugs. Even when killing just 5% of the surviving mutants, the detection probability remains substantial at 63.7%. These results indicate that even minimal mutant-killing effort can substantially strengthen a test suite’s ability to detect real bugs.

These results naturally raise the question of why even minimal mutant-killing effort can be so effective. Our analysis indicates that the benefits of mutant killing do not hinge on identifying a specific mutant or a fortuitous assertion. On average, each of the 63 real bugs can be exposed by killing any of 21 different mutants (*RQ3*), where each such mutant induces at least one assertion capable of detecting the real bug. This finding contrasts sharply with prior results by Just *et al.* [35], who found that a real bug is coupled, on average, with only two mutants in an earlier version of DEFECTS4J. Such prior empirical approximations of coupling relationships are established through one or a small number of documented bug-fixing tests, where the test was designed with prior knowledge of the real-bug. However, consider that there might be other tests and assertions that may detect and fix the bug. By being blind to a specific bug, and in trying to kill more than one mutant, a developer may be exposed to a more open-ended space of test-assertions and test-designs. We found more coupled mutants in our work (21 vs. 2), because we explored a broader space of test-assertions, guided by multiple mutant-killing options.

Also note that our results indicate that killing a mutant deemed “coupled” through a specific triggering test may not necessarily detect the real-bug. We know that such mutants are “coupled” because our technique generated at least one assertion (out of many assertions induced by the mutant) that will also detect the real-bug. However, under the constraint of selecting only a single, newly generated assertion per mutant, and even when killing 100% of the “coupled” mutants, the

real-bug detection probability reaches only 95.9%. Known coupling-effect(s) between a mutant and real-bug does not automatically lend itself to generating bug-detecting tests and assertions.

We offer the practical takeaway that real-bug detection arises from exploiting multiple, overlapping mutant-killing opportunities. This takeaway, along with our finding of bug-detection effectiveness when killing even 5% of surviving mutants, further motivates mutant-sampling, -prioritization, and -selection techniques, which aim to reduce mutation-analysis cost while preserving much of its test-improvement potential.

How Mutant Killing Translates into Real-Bug Detection. The operation of our technique to generate assertions is not influenced by the nature or source of the fault (mutant or real-bug), for which it is generating new assertions. This offers us a common, unified framework to compare mutant-derived and real-bug-derived assertions and their fine-grained runtime characteristics. Our results when performing comparisons indicate that real bugs generally exhibit broader runtime-behavioral impact than killable surviving mutants. Specifically, as reported in RQ5, real bugs affect more memory locations (11.5 vs. 2.0), variables (6.0 vs. 2.0), and tests (4.0 vs. 1.0), and involve more program-output attributes through shallower access paths (2.4 vs. 3.8). Despite this disparity, we observe that approximately 31% of assertions derived from mutants are nevertheless capable of detecting the corresponding real bugs.

When limiting our observations to bug-detecting assertions generated from coupled mutants, our analysis shows that 40.6% of these assertions are identical to assertions derivable from real bugs. These cases correspond to situations in which the mutant and the real bug induce state infections at the same memory locations. Another substantial fraction (41.3%) of these mutant-derived assertions validate only a strict subset of memory locations that are otherwise inspected by assertions when derived from real-bugs. In other words, when mutation-based test improvements succeed in detecting real bugs, they often do so by validating a narrower slice of a broader program state that is impacted by those bugs. These observations help explain how killing relatively simple mutants can nevertheless contribute to real-bug detection. Under this interpretation of the coupling effect hypothesis, tests designed to kill simple surviving mutants may coincidentally validate portions of program state impacted by real bugs, thereby exposing those bugs even without necessarily capturing their broader effects.

When Bugs Cannot Be Detected in Our Experiments. Our analysis suggests that the non-detectability of the remaining 41 out of 104 real bugs is primarily attributable to limitations in the mutation operators used. In particular, the operator set provided by Major fails to generate surviving mutants that induce assertions capable of detecting real bugs. As noted by Gay *et al.* [25], Major offers a limited but core set of mutation operators compared to tools such as MuJava [44] or PIT [11]. While using Major reflects the selective and cost-conscious nature of mutation analysis in real-world settings, it also highlights the importance of mutation operator diversity in producing test improvements that align with real-bug behavior. For this purpose, in our accompanying experiment artifact we provide a detailed analysis of the contributions of different mutation operators. This finding motivates future work to understand the relative value of different mutation operators and their role in supporting effective mutant killing and meaningful test improvement.

Subsumption among Coupled Mutants. When multiple mutants couple to the same real bug, a natural question is whether these coupled mutants represent independent detection opportunities or largely subsume one another. Prior work inferred empirical subsumption from the failing-test sets of killed mutants [51], an approach that cannot be applied to surviving mutants because they have no failing tests. Our fine-grained analysis, however, identifies every memory location at which a mutant's infection can be revealed with mutant-killing assertions, enabling subsumption analysis for surviving mutants. Treating each derivable assertion as a potential failing (mutant killing) test,

we consider mutant m_1 to subsume m_2 if every assertion derivable from m_1 also kills m_2 ; under this definition, 93.7% of coupled mutants are subsumed by another coupled mutant, consistent with the substantial subsumption previously reported among killed mutants [51]. Thus, these coupled mutants exhibit substantial behavioral overlaps. However, as with empirical approaches to investigating the concept of “coupling,” these subsumption analyses rely on test-dependent empirical approximations rather than semantic subsumption relations that hold universally at the mutant or operator level. Nevertheless, the substantial overlap among mutant behaviors helps explain why multiple mutants can provide repeated opportunities to strengthen tests and thereby detect or prevent real bugs. Meanwhile, this overlap introduces considerable redundancy and motivates future work on mutant selection and mutant subsumption analysis.

8 Threats to Validity

Internal Validity. The primary threat to internal validity arises from treating assertion-augmented mutant killing as a causal intervention on the test suite. However, our technique isolates fault effects at a fine granularity by localizing each augmentation to a single assertion. As such, the controlled design supports principled causal reasoning about the effect of mutant killing on real-bug detection. Another threat stems from potential non-determinism introduced by generated assertions. Such non-determinism may cause flakiness due to runtime variability or non-deterministic program state [43, 61]. To mitigate these effects and support replicability, we execute each test in an isolated Java Virtual Machine, reducing order-dependent flakiness. Each generated fault-revealing assertion is validated individually and accepted only after passing a minimum seven-run execution pipeline.

External Validity. Our experiments rely on the DEFECTS4J dataset, which contains a limited number of real bugs and focuses on Java programs. As such, the results may not generalize to other datasets or programming languages. However, DEFECTS4J is a widely studied real-bug benchmark that has been investigated for over a decade [34], facilitating direct comparison with prior work.

Moreover, we use Major to generate mutants, whereas other tools (e.g., PIT [11] and MuJava [44]) offer different mutation operators that may yield different results. However, Major focuses on traditional, simple syntactic mutations, which aligns with the coupling hypothesis. Plus, Major is widely used and tightly integrated with DEFECTS4J, thereby supporting replication and comparability.

Finally, we focus on assertion augmentation as the means of killing surviving mutants. Thus, our findings apply specifically to this class of mutant-killing behaviors and may not generalize to other approaches. Although our framework for modeling and quantifying mutant-killing effectiveness is broadly applicable to other mutant-killing approaches, such approaches may not provide the same controlled, narrowly scoped instrument for investigating the mechanisms underlying the coupling effect. We envision two complementary empirical directions for obtaining broader, potentially more generalizable evidence on mutant coupling using other mutant-killing approaches. First, a human observational study could ask participants to write tests that kill pre-identified coupled mutants in a fixed program and then evaluate whether those tests also detect the corresponding real bugs. This process might be affordable in a teaching-oriented setting with DEFECTS4J, and the resulting observations would complement our findings with evidence from human mutant-killing practices in educational contexts. Second, LLM-based techniques could, in principle, target arbitrary surviving mutants. However, their trial-and-error nature, instability across repeated runs, and tendency to produce non-compilable or noisy outputs make fine-grained causal attribution difficult. Such techniques may therefore trade some of the mechanistic insight afforded by our controlled analysis for broader coverage of mutant-killing practices.

Construct Validity. Threats to construct validity primarily stem from how we operationalize mutant killing. In this study, we model mutant killing exclusively through assertion augmentation,

whereas mutation testing in practice may also involve other activities, such as test-input generation, test refactoring, and production-code changes [8, 17, 55], which requires further investigation in practical scenarios. In addition, our modeling of budgeted, incremental mutant-killing workflows may not fully capture how practitioners prioritize mutants or select assertions in real-world practice. Indeed, how developers organically kill individual surviving mutants in practice remains largely unknown, owing to the limited empirical evidence from real-world production settings. As such, future research should investigate the prevalence and effects of broader mutant-killing practices.

9 Related Work

Insights from Prior Studies on Mutant Killing. Mutation testing, introduced by DeMillo *et al.* in the late 1970s [15], builds on the coupling effect and competent programmer hypotheses [32]. Most existing empirical evidence on the value of mutant killing activities is observational in nature. Prior work includes practitioner *surveys* by Sánchez *et al.* [60], *literature reviews* on the role of mutation testing in quality assurance by Zhu *et al.* [71], *experience reports* in the Linux kernel by Ahmed *et al.* [3], as well as *case studies* conducted in industrial settings such as the nuclear domain by Delgado-Pérez *et al.* [14] and in academic environments by van Heijningen *et al.* [66].

Many studies examined *correlational* relationships between mutant detection and complex faults. Offutt *et al.* [47, 48] first examined whether tests killing simple mutants can detect complex mutants, which was later formalized as higher-order mutants by Jia *et al.* [31]. Subsequent empirical studies (e.g., [57]) compared the effectiveness of test suites against real faults, hand-seeded faults, and mutants. Moreover, Namin *et al.* [46] later examined additional factors affecting mutation score metrics, including mutation operators, test-suite size, and programming language.

More recent studies have shifted their focus toward real bugs. Just *et al.* [35] curated the DEFECTS4J real-bug dataset [34] and investigated whether tests that detect real bugs also kill additional mutants, which they considered to be coupled to real bugs. Laurent *et al.* [40] later revisited this analysis on DEFECTS4J 2.0 using the same methodology. Complementary work by Ojdanic *et al.* [50] compared syntactic and semantic similarities between mutants and real faults based on test-execution behavior, while Gay *et al.* [25] proposed a graded notion of coupling by quantifying similarity through overlapping and non-overlapping triggering tests and their underlying failure causes.

Large-scale industrial studies have investigated mutant-killing behavior using proprietary datasets [8, 54, 55]. In particular, Google [55] analyzed developer responses to surviving mutants surfaced during code reviews, examining how subsequent test and code changes correlate with mutant resolution in the industrial setting. In a separate study, the same authors employed a quasi-interventional design to evaluate whether mutation testing would have surfaced mutants that were later killed by bug-fixing tests [54].

In contrast, our work conducts a controlled intervention experiment that models surviving-mutant killing under a realistic mutant-killing budget and directly measures its impact on real-bug detection. This design enables us to assess whether, and to what extent, mutant killing uncovers previously unseen real bugs, and to examine the mechanisms through which mutant killing translates into real-bug detection, as posited by the coupling-effect hypothesis.

Mutant Killing Approaches. Surviving-mutant killing can be achieved through a variety of approaches [17]. As reported by Sánchez *et al.* [60], practitioners pursue different goals when killing surviving mutants, including using new test inputs and strengthening test oracles. Evidence from industrial settings further indicates that developers may also modify production code — beyond tests alone — when resolving surfaced mutants, as shown in Google’s analysis [55].

Among approaches that generate or improve test assertions, the closest related work is by Du *et al.* [19], which derives assertion-amplification specifications to kill surviving mutants via propagated

infection. Strengthening test oracles is an instance of the long-standing *oracle problem* [6], for which prior work has explored machine-learning-based [21] and crowd-based oracle generation [53]. Similarly, Vera-Pérez *et al.* [67] propose mutation operators that suggest targeted test improvements, while Gay *et al.* [26] and Staats *et al.* [63] introduce mutation-based analyses to support oracle data selection. These approaches primarily provide guidance or suggestions for assertion generation and typically require developer involvement to finalize tests. In contrast, other work generates assertions holistically as part of whole test-suite generation or assertion augmentation [23, 68], whereas we develop a fully automated technique that kills specific mutants.

Beyond assertion-based techniques, mutant killing has also been studied through test-input generation approaches. For example, SEMu [65] employs dynamic symbolic execution to generate inputs targeting specific surviving mutants. However, symbolic-execution-based techniques are typically evaluated on C benchmarks such as GNU Coreutils [65], which limits their applicability to Java-based real-bug datasets. Other techniques aim for full test suite generation, including whole test-suite generation approaches such as EvoSuite [22], methods from the search-based software testing community [4, 23, 24, 59, 62], and test-amplification frameworks such as DSpot [7, 13] and EvoSuite_{AMP} [57]. However, these techniques generally produce non-targeted test augmentations, in contrast to our approach, which focuses on selectively killing specific surviving mutants.

Recently, large language models (LLMs) have enabled new approaches to test generation and mutant killing. For example, Straubinger *et al.* [64] propose an iterative, LLM-driven approach inspired by scientific debugging that kills specific mutants to guide test generation. Meta has also explored LLM-based frameworks for mutation analysis and mutant killing [29]. LLMs have also been applied to test-oracle generation directly, *e.g.*, AUGMENTEST derives oracles from natural-language documentation [39]; the many neural and LLM-based oracle-automation techniques cataloged in a recent survey [45] further illustrate this trend. However, LLM non-determinism and the potential exposure to real-bug information complicate their use in controlled analyses aimed at isolating the effects of mutant killing as compared to our technique.

10 Conclusion

This work directly examines a long-standing assumption in mutation testing: that killing surviving mutants helps detect real bugs, which we study on the subset of mutants that survive due to revealability gaps. By modeling mutant killing as a selective, incremental activity and restricting test improvements to attributable assertion augmentation, we isolate and measure its causal impact on real-bug detection. Our controlled experiment provides concrete evidence that mutant killing can meaningfully strengthen test suites, even under realistic budget constraints, and clarifies the mechanisms through which mutants act as effective proxies for real faults in our setting. Future work includes extending this controlled evaluation framework beyond assertion augmentation to other forms of test and code improvements, and studying how developer-guided prioritization strategies further shape the effectiveness of mutant killing in practice.

11 Data Availability

To foster transparency and reproducibility, we publicly archive the source code and data used in this study on Zenodo [16]. The accompanying artifact repository, including additional mutation-operator analyses, is maintained on GitHub at https://github.com/spideruci/Defects4j_for_Coupling_Effect_Analysis_Through_Killing_Surviving_Mutants.

Acknowledgments

The second author's opinions expressed in this publication are solely his and do not purport to reflect the opinions or views of his employer, Microsoft.

References

- [1] 2015. HelpFormatter.setArgName() has no effect. <https://issues.apache.org/jira/browse/CLI-205>
- [2] 2026. JavaParser. <https://javaparser.org/>
- [3] Iftekhar Ahmed, Carlos Jensen, Alex Groce, and Paul E. McKenney. 2017. Applying Mutation Analysis on Kernel Test Suites: An Experience Report. In *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 110–115. doi:10.1109/ICSTW.2017.26
- [4] Hussein Almulla and Gregory Gay. 2022. Learning how to search: generating effective test cases through adaptive fitness function selection. *Empirical Software Engineering* 27, 2 (2022), 38. doi:10.1007/s10664-021-10048-8
- [5] James Andrews, Lionel Briand, Yvan Labiche, and Akbar Siami Namin. 2006. Using Mutation Analysis for Assessing and Comparing Testing Coverage Criteria. *Software Engineering, IEEE Transactions on* 32 (09 2006), 608–624. doi:10.1109/TSE.2006.83
- [6] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. doi:10.1109/TSE.2014.2372785
- [7] Benoit Baudry, Simon Allier, Marcelino Rodriguez-Cancio, and Martin Monperrus. 2015. DSpot: Test Amplification for Automatic Assessment of Computational Diversity. (03 2015). <http://arxiv.org/abs/1503.05807>
- [8] Moritz Beller, Chu-Pan Wong, Johannes Bader, Andrew Scott, Mateusz Machalica, Satish Chandra, and Erik Meijer. 2021. What it would take to use mutation testing in industry—a study at facebook. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 268–277. doi:10.1109/ICSE-SEIP52600.2021.00036
- [9] E. Bruneton, R. Lenglet, and T. Coupaye. 2002. ASM: A code manipulation tool to implement adaptable systems. In *Adaptable and Extensible Component Systems*.
- [10] Zimin Chen, Małgorzata Salawa, Manushree Vijayvergiya, Goran Petrović, Marko Ivanković, and René Just. 2023. MuRS: Mutant Ranking and Suppression using Identifier Templates. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (, San Francisco, CA, USA,) (ESEC/FSE 2023). Association for Computing Machinery, New York, NY, USA, 1798–1808. doi:10.1145/3611643.3613901
- [11] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. Pit: a practical mutation testing tool for java. In *Proceedings of the 25th international symposium on software testing and analysis*. 449–452. doi:10.1145/2931037.2948707
- [12] Benjamin Danglot, Oscar Vera-Perez, Zhongxing Yu, Andy Zaidman, Martin Monperrus, and Benoit Baudry. 2019. A snowballing literature study on test amplification. *Journal of Systems and Software* 157 (2019), 110398. doi:10.1016/j.jss.2019.110398
- [13] Benjamin Danglot, Oscar Luis Vera-Pérez, Benoit Baudry, and Martin Monperrus. 2019. Automatic test improvement with DSpot: a study with ten mature open-source projects. *Empirical Software Engineering* 24 (2019), 2603–2635. doi:10.1007/s10664-019-09692-y
- [14] Pedro Delgado-Pérez, Ibrahim Habli, Steve Gregory, Rob Alexander, John Clark, and Inmaculada Medina-Bulo. 2018. Evaluation of Mutation Testing in a Nuclear Industry Case Study. *IEEE Transactions on Reliability* 67, 4 (2018), 1406–1419. doi:10.1109/TR.2018.2864678
- [15] Richard A DeMillo, Richard J Lipton, and Frederick G Sayward. 1978. Hints on test data selection: Help for the practicing programmer. *Computer* 11, 4 (1978), 34–41. doi:10.1109/C-M.1978.218136
- [16] Hang Du, Vijay Krishna Palepu, and James Jones. 2026. *Artifact for ISSTA 2026 paper: How Does Killing Surviving Mutants Help Detect Real Bugs with Assertion Generation? A Controlled Experiment*. doi:10.5281/zenodo.21328338
- [17] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2023. To Kill a Mutant: An Empirical Study of Mutation Testing Kills. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 715–726. doi:10.1145/3597926.3598090
- [18] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2024. Ripples of a Mutation — An Empirical Study of Propagation Effects in Mutation Testing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 116, 13 pages. doi:10.1145/3597503.3639179
- [19] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2025. Leveraging Propagated Infection to Crossfire Mutants . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 3136–3148. doi:10.1109/ICSE55347.2025.00150
- [20] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2026. Revealing Regressions: A Comparative Study of State-Capture Strategies in Validating Program Behavior. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE079 (June 2026), 24 pages. doi:10.1145/3797107
- [21] Afonso Fontes and Gregory Gay. 2021. Using machine learning to generate test oracles: a systematic literature review. In *Proceedings of the 1st International Workshop on Test Oracles* (Athens, Greece) (TORACLE 2021). Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3472675.3473974

- [22] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering* (Szeged, Hungary) (ESEC/FSE '11). Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [23] Gordon Fraser and Andrea Arcuri. 2013. Whole Test Suite Generation. *IEEE Transactions on Software Engineering* 39, 2 (2013), 276–291. doi:10.1109/TSE.2012.14
- [24] Gordon Fraser and Andrea Arcuri. 2015. Achieving scalable mutation-based generation of whole test suites. *Empirical Software Engineering* 20, 3 (2015), 783–812. doi:10.1007/s10664-013-9299-z
- [25] Gregory Gay and Alireza Salahi. 2023. How Closely are Common Mutation Operators Coupled to Real Faults?. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 129–140. doi:10.1109/ICST57152.2023.00021
- [26] Gregory Gay, Matt Staats, Michael Whalen, and Mats P. E. Heimdahl. 2015. Automated Oracle Data Selection Support. *IEEE Transactions on Software Engineering* 41, 11 (2015), 1119–1137. doi:10.1109/TSE.2015.2436920
- [27] Rahul Gopinath, Iftekhar Ahmed, Mohammad Amin Alipour, Carlos Jensen, and Alex Groce. 2017. Mutation Reduction Strategies Considered Harmful. *IEEE Transactions on Reliability* 66, 3 (2017), 854–874. doi:10.1109/TR.2017.2705662
- [28] Rahul Gopinath, Mohammad Amin Alipour, Iftekhar Ahmed, Carlos Jensen, and Alex Groce. 2016. On the limits of mutation reduction strategies. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) (ICSE '16). Association for Computing Machinery, New York, NY, USA, 511–522. doi:10.1145/2884781.2884787
- [29] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. 2025. Mutation-Guided LLM-based Test Generation at Meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) (FSE Companion '25). Association for Computing Machinery, New York, NY, USA, 180–191. doi:10.1145/3696630.3728544
- [30] Jerry L. Hintze and Ray D. Nelson. 1998. Violin Plots: A Box Plot-Density Trace Synergism. *The American Statistician* 52, 2 (1998), 181–184. arXiv:https://www.tandfonline.com/doi/pdf/10.1080/00031305.1998.10480559 doi:10.1080/00031305.1998.10480559
- [31] Yue Jia and Mark Harman. 2009. Higher Order Mutation Testing. *Information and Software Technology* 51, 10 (2009), 1379–1393. Source Code Analysis and Manipulation, SCAM 2008. doi:10.1016/j.infsof.2009.04.016
- [32] Yue Jia and Mark Harman. 2010. An analysis and survey of the development of mutation testing. *IEEE transactions on software engineering* 37, 5 (2010), 649–678. doi:10.1109/TSE.2010.62
- [33] Rena Just. 2024. Mutation analysis does not support 3 gson and 19 mockito bugs - issue #615 - RJUST/DEFECTS4J. https://github.com/rjust/defects4j/issues/615
- [34] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440. doi:10.1145/2610384.2628055
- [35] René Just, Darioush Jalali, Laura Inozemtseva, Michael D Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 654–665. doi:10.1145/2635868.2635929
- [36] René Just, Bob Kurtz, and Paul Ammann. 2017. Inferring mutant utility from program context. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 284–294. doi:10.1145/3092703.3092732
- [37] René Just, Franz Schweiggert, and Gregory M Kapfhammer. 2011. MAJOR: An efficient and extensible tool for mutation analysis in a Java compiler. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 612–615. doi:10.1109/ASE.2011.6100138
- [38] Samuel J. Kaufman, Ryan Featherman, Justin Alvin, Bob Kurtz, Paul Ammann, and René Just. 2022. Prioritizing Mutants to Guide Mutation Testing. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 1743–1754. doi:10.1145/3510003.3510187
- [39] Shaker Mahmud Khandaker, Fitsum Kifetew, Davide Prandi, and Angelo Susi. 2025. AugmenTest: Enhancing Tests with LLM-Driven Oracles. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 279–289. doi:10.1109/ICST62969.2025.10988926
- [40] Thomas Laurent, Stephen Gaffney, and Anthony Ventresque. 2022. Re-visiting the coupling between mutants and real faults with Defects4J 2.0. In *2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 189–198. doi:10.1109/ICSTW55395.2022.00042
- [41] Nan Li and Jeff Offutt. 2017. Test Oracle Strategies for Model-Based Testing. *IEEE Transactions on Software Engineering* 43, 4 (2017), 372–395. doi:10.1109/TSE.2016.2597136
- [42] Birgitta Lindstrom and András Miki. 2016. On strong mutation and subsuming mutants. In *2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 112–121. doi:10.1109/ICSTW.2016.28
- [43] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) (FSE 2014). Association for Computing Machinery, New York, NY, USA, 643–653. doi:10.1145/2635868.2635920

- [44] Yu-Seung Ma, Jeff Offutt, and Yong-Rae Kwon. 2006. MuJava: a mutation system for java. In *Proceedings of the 28th International Conference on Software Engineering* (Shanghai, China) (ICSE '06). Association for Computing Machinery, New York, NY, USA, 827–830. doi:10.1145/1134285.1134425
- [45] Facundo Molina, Alessandra Gorla, and Marcelo d'Amorim. 2025. Test Oracle Automation in the Era of LLMs. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 150 (May 2025), 24 pages. doi:10.1145/3715107
- [46] Akbar Siami Namin and Sahitya Kakarla. 2011. The Use of Mutation in Testing Experiments and Its Sensitivity to External Threats. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis* (Toronto, Ontario, Canada) (ISSTA '11). Association for Computing Machinery, New York, NY, USA, 342–352. doi:10.1145/2001420.2001461
- [47] A. Offutt. 1989. The coupling effect: fact or fiction. In *Proceedings of the ACM SIGSOFT '89 Third Symposium on Software Testing, Analysis, and Verification* (Key West, Florida, USA) (TAV3). Association for Computing Machinery, New York, NY, USA, 131–140. doi:10.1145/75308.75324
- [48] A. Jefferson Offutt. 1992. Investigations of the software testing coupling effect. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (January 1992), 5–20. doi:10.1145/125489.125473
- [49] A Jefferson Offutt and Roland H Untch. 2001. Mutation 2000: Uniting the orthogonal. *Mutation testing for the new century* (2001), 34–44. doi:10.1007/978-1-4757-5939-6_7
- [50] Milos Ojdanic, Aayush Garg, Ahmed Khanfir, Renzo Degiovanni, Mike Papadakis, and Yves Le Traon. 2023. Syntactic Versus Semantic Similarity of Artificial and Real Faults in Mutation Testing Studies. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3922–3938. doi:10.1109/TSE.2023.3277564
- [51] Mike Papadakis, Christopher Henard, Mark Harman, Yue Jia, and Yves Le Traon. 2016. Threats to the Validity of Mutation-Based Test Assessment. In *Proceedings of the 25th International Symposium on Software Testing and Analysis* (Saarbrücken, Germany) (ISSTA 2016). Association for Computing Machinery, New York, NY, USA, 354–365. doi:10.1145/2931037.2931040
- [52] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Chapter Six - Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, Vol. 112. Elsevier, 275–378. doi:10.1016/bs.adcom.2018.03.015
- [53] Fabrizio Pastore, Leonardo Mariani, and Gordon Fraser. 2013. CrowdOracles: Can the Crowd Solve the Oracle Problem?. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*. 342–351. doi:10.1109/ICST.2013.13
- [54] Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. 2021. Does mutation testing improve testing practices?. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 910–921. doi:10.1109/ICSE43902.2021.00087
- [55] Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. 2023. Please fix this mutant: How do developers resolve mutants surfaced during code review?. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 150–161. doi:10.1109/ICSE-SEIP58684.2023.00019
- [56] Md Nakhla Rafi, An Ran Chen, Tse-Hsun Peter Chen, and Shaohua Wang. 2025. Revisiting Defects4J for Fault Localization in Diverse Development Scenarios. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 63–75. doi:10.1109/MSR66628.2025.00022
- [57] Muhammad Firhard Roslan, José Miguel Rojas, and Phil McMinn. 2022. An Empirical Comparison of EvoSuite and DSpot for Improving Developer-Written Test Suites with Respect to Mutation Score. In *Search-Based Software Engineering*. Mike Papadakis and Silvia Regina Vergilio (Eds.). Springer International Publishing, Cham, 19–34. doi:10.1007/978-3-031-21251-2_2
- [58] Sheldon M. Ross. 2020. *A first course in probability* (10th global ed.). Pearson, Harlow, United Kingdom.
- [59] Alireza Salahirad, Hussein Almulla, and Gregory Gay. 2019. Choosing the fitness function for the job: Automated generation of test suites that detect real faults. *Software Testing, Verification and Reliability* 29 (06 2019), e1701. doi:10.1002/stvr.1701
- [60] Ana B. Sánchez, José A. Parejo, Sergio Segura, Amador Durán, and Mike Papadakis. 2024. Mutation Testing in Practice: Insights From Open-Source Software Developers. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1130–1143. doi:10.1109/TSE.2024.3377378
- [61] August Shi, Jonathan Bell, and Darko Marinov. 2019. Mitigating the effects of flaky tests on mutation testing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 112–122. doi:10.1145/3293882.3330568
- [62] Rodolfo Adamshuk Silva, Simone do Rocio Senger de Souza, and Paulo Sérgio Lopes de Souza. 2017. A systematic review on search based mutation testing. *Information and Software Technology* 81 (2017), 19–35. doi:10.1016/j.infsof.2016.01.017
- [63] Matt Staats, Gregory Gay, and Mats PE Heimdahl. 2012. Automated oracle creation support, or: How I learned to stop worrying about fault propagation and love mutation testing. In *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 870–880. doi:10.1109/ICSE.2012.6227132

- [64] Philipp Straubinger, Marvin Kreis, Stephan Lukasczyk, and Gordon Fraser. 2025. Mutation Testing via Iterative Large Language Model-Driven Scientific Debugging. [doi:10.48550/arXiv.2503.08182](https://doi.org/10.48550/arXiv.2503.08182)
- [65] Titcheu Chekam Thierry, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2021. Killing Stubborn Mutants with Symbolic Execution. *ACM Transactions on Software Engineering and Methodology* 30 (01 2021), 1–23. [doi:10.1145/3425497](https://doi.org/10.1145/3425497)
- [66] Stefan Alexander van Heijningen, Theo Wiik, Francisco Gomes de Oliveira Neto, Gregory Gay, Kim Viggedal, and David Friberg. 2024. Integrating Mutation Testing Into Developer Workflow: An Industrial Case Study. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 2110–2120. [doi:10.1145/3691620.3695273](https://doi.org/10.1145/3691620.3695273)
- [67] Oscar Luis Vera-Pérez, Benjamin Danglot, Martin Monperrus, and Benoit Baudry. 2019. Suggestions on Test Suite Improvements with Automatic Infection and Propagation Analysis. arXiv:1909.04770 [cs.SE] <https://arxiv.org/abs/1909.04770>
- [68] Tao Xie. 2006. Augmenting automatically generated unit-test suites with regression oracle checking. In *Proceedings of the 20th European Conference on Object-Oriented Programming* (Nantes, France) (ECOOP'06). Springer-Verlag, Berlin, Heidelberg, 380–403. [doi:10.1007/11785477_23](https://doi.org/10.1007/11785477_23)
- [69] Xiangjuan Yao, Mark Harman, and Yue Jia. 2014. A study of equivalent and stubborn mutation operators using human analysis of equivalence. In *Proceedings of the 36th international conference on software engineering*. 919–930. [doi:10.1145/2568225.2568265](https://doi.org/10.1145/2568225.2568265)
- [70] Peng Zhang, Yang Wang, Xutong Liu, Yanhui Li, Yibiao Yang, Ziyuan Wang, Xiaoyu Zhou, Lin Chen, and Yuming Zhou. 2022. Mutant Reduction Evaluation: What is There and What is Missing? *ACM Trans. Softw. Eng. Methodol.* 31, 4, Article 69 (jul 2022), 46 pages. [doi:10.1145/3522578](https://doi.org/10.1145/3522578)
- [71] Qianqian Zhu, Annibale Panichella, and Andy Zaidman. 2018. A systematic literature review of how mutation testing supports quality assurance processes. *Software Testing, Verification and Reliability* 28, 6 (2018), e1675. [doi:10.1002/stvr.1675](https://doi.org/10.1002/stvr.1675)

Received 2026-01-29; accepted 2026-04-16